

SOLUTION BRIEF

# Vault→Pod Drift Tracing

Autonomous causation tracing from missing Vault keys to failing GPU workloads — in-cluster, without SaaS egress

|                                 |                              |                                     |                               |
|---------------------------------|------------------------------|-------------------------------------|-------------------------------|
| Works with existing Vault + ESO | No telemetry egress required | Conservative write permissions only | Training & inference clusters |
|---------------------------------|------------------------------|-------------------------------------|-------------------------------|

## Executive Summary

GPU clusters running HashiCorp Vault and External Secrets Operator share a common, underserved failure class: configuration drift between secret definitions in Vault and the consuming workloads in Kubernetes. This drift goes undetected for hours or days. Existing monitoring surfaces these clusters as “degraded” but cannot identify the precise cause or trace the chain from a missing Vault key to the failing pod.

This solution describes how to instrument that causation chain autonomously — walking each layer of the Vault→ESO→Kubernetes Secret→pod binding pipeline, identifying where the chain breaks, and remediating with conservative write permissions — entirely in-cluster with no telemetry sent to a third-party SaaS.

**The core gap**

No existing tool traces the complete chain: Vault key path → ExternalSecret CRD → Kubernetes Secret → pod env/volume mount → pod failure. HashiCorp VSO detects store→K8s drift but not K8s→pod causation. ArgoCD intentionally ignores ESO-managed secrets. Datadog surfaces the symptom (pod degraded) not the cause. This brief describes the diagnostic engine that fills that gap.

## The Vault→Pod Chain: What It Is and Where It Breaks

Every secret-dependent workload traverses five distinct layers between the authoritative secret store and the running container. Drift can occur at any layer. The problem is not any single layer breaking — it is that no tool currently joins all five layers into one diagnostic object.

| Step | Layer                            | Object                             | How drift occurs                                                           | Workload impact                                 |
|------|----------------------------------|------------------------------------|----------------------------------------------------------------------------|-------------------------------------------------|
| 1    | Vault (secret store)             | KV path / Policy                   | Key deleted, rotated, or policy revoked without updating ESO reference     | ESO sync fails silently or returns 404          |
| 2    | ExternalSecret CRD               | spec.data[].remoteRef.key          | Path in spec points to deleted or renamed Vault key                        | K8s Secret not created or stale value persists  |
| 3    | SecretStore / ClusterSecretStore | auth credentials / refreshInterval | ServiceAccount token expires; refreshInterval too long to catch rotation   | All ExternalSecrets on this store fail together |
| 4    | Kubernetes Secret                | data[] / metadata                  | Secret exists but value is empty, wrong key, or base64 misencoded          | Pod mounts stale or empty credential            |
| 5    | Pod spec                         | env.valueFrom / volumeMount        | Pod references wrong Secret name or key; Secret name drifted from manifest | CrashLoopBackOff or ImagePullBackOff at init    |

The critical insight is that failures at layer 1 or 2 take minutes to appear at layer 5 — and monitoring tools only see layer 5. By the time the SRE war room opens, the pod has been crashing for long enough to consume significant GPU idle time. For a 96-node H100 cluster at \$3/GPU/hour, even a 2-hour diagnostic gap costs over \$500 in wasted reserved compute.

## What “Trace Vault→Pod Drift” Actually Means

Tracing is not alerting. An alert says “pod is unhealthy.” Tracing answers: “which Vault path is missing, which ExternalSecret references it, which Kubernetes Secret failed to populate, which pod is consuming it, and what is the precise error at each hop.”

### Layer 1: Vault key existence and policy check

The tracer authenticates to Vault using the cluster’s existing service account (the same identity ESO uses) and verifies that each key path referenced in deployed ExternalSecrets exists and is readable under the current policy. It does not require Vault admin credentials — only the read permission

← vault\_pod\_drift\_solution\_bri...

(API)

Policy denies read permission for the ESO service account  
Key exists but version mismatch between ExternalSecret spec and Vault KV v2 metadata

**Layer 2: ExternalSecret → SecretStore binding verification**

The tracer enumerates all ExternalSecret resources in the cluster and cross-references each spec.data[].remoteRef against the verified Vault key inventory from layer 1. Mismatches are flagged as broken references. The tracer also validates that the referenced SecretStore or ClusterSecretStore is healthy and that its authentication credentials (typically a Kubernetes ServiceAccount with Vault auth) are current and non-expired.

**Why this layer is uniquely dangerous**

ESO's own status conditions (Ready/NotReady) only reflect whether the last sync attempt succeeded. If the sync interval is 1 hour and a Vault key was deleted 5 minutes after the last successful sync, the ExternalSecret will show status Ready for up to 59 more minutes — while the Kubernetes Secret it manages is already serving a stale credential to every pod that mounts it.

**Layer 3: Kubernetes Secret integrity check**

The tracer reads the Kubernetes Secret that the ExternalSecret is expected to have populated and validates: (a) the Secret exists, (b) each expected key is present and non-empty, (c) the value hash matches what Vault would return for the current version of the key, and (d) the Secret's metadata annotations show a recent reconcile timestamp consistent with the configured refresh interval.

|                                                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Drift states detected</b></p> <ul style="list-style-type: none"><li>• Secret exists, data key missing</li><li>• Secret value is empty string</li><li>• Last-applied-configuration annotation stale beyond 2× refresh interval</li><li>• Value hash diverged from Vault source (rotation not propagated)</li></ul> | <p><b>Remediation actions (conservative)</b></p> <ul style="list-style-type: none"><li>• Trigger manual ESO refresh on the specific ExternalSecret</li><li>• Annotate the Secret to force reconciliation without deleting it</li><li>• Alert with full chain context — Vault path, ExternalSecret name, Secret name, affected pods</li><li>• Never modify Vault contents; never delete Kubernetes Secrets</li></ul> |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Layer 4: Pod→Secret binding resolution**

The tracer walks every running pod's spec and resolves each env.valueFrom.secretKeyRef and each volume mount of type secret against the Secret inventory verified in layer 3. It constructs a bipartite graph: secrets on one side, pods on the other, with edges representing declared dependencies. Any pod with an edge to a degraded or missing secret is flagged as "drift-affected" before it crashes — enabling pre-emptive remediation.

For GPU workloads specifically, the tracer also inspects the NVIDIA device plugin annotations and any init container specs (common in DGX stack deployments for model pre-pull steps) that pull credentials from the same Secret chain. A single drift event at layer 1 can cascade through multiple init containers and the main training container simultaneously.

**Layer 5: Diagnostic event emission**

When the tracer detects a broken chain at any layer, it emits a structured Kubernetes Event and an optional alert payload with the complete diagnostic object:

#### Example diagnostic event payload

```
vault_path:      secret/data/ml-platform/hf-token  [NOT FOUND]
external_secret: ml-namespace/hf-token-sync  [Sync failed: 404]
k8s_secret:      ml-namespace/hf-credentials  [key hf_token empty]
affected_pods:   training-job-7x9k2, training-job-7x9k3  [CrashLoopBackOff]
root_cause:      Vault key deleted 14m ago; ESO refresh interval 1h; pods crashing
                  since 12m
remediation:     ExternalSecret manual refresh triggered; pods will restart with
                  valid credential
```

## Architecture: In-Cluster, No Egress

The tracer runs as a Kubernetes controller (Deployment) inside the target cluster. It watches ExternalSecret and Secret resources via the Kubernetes informer framework and performs Vault checks using the cluster's own service account token. All diagnostic data is stored as Kubernetes Events and CRDs within the cluster. Nothing leaves the cluster boundary.

| Component                         | Function                                                                               | RBAC requirements                                                                 |
|-----------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Drift Tracer controller</b>    | Runs reconcile loop over all ExternalSecrets; builds chain graph; detects breaks       | Read: ExternalSecrets, Secrets, Pods, Events. Write: Events, DriftReport CRD only |
| <b>Vault probe sidecar</b>        | Authenticates to Vault via K8s ServiceAccount; validates key path existence and policy | Same ServiceAccount already granted to ESO — no new Vault permissions needed      |
| <b>DriftReport CRD</b>            | Stores structured diagnostic objects per affected chain; queryable via kubectl         | Create/Update on DriftReport resources only                                       |
| <b>Alert forwarder (optional)</b> | Sends DriftReport payload to Alertmanager, PagerDuty, or Slack webhook — configurable  | No cluster egress required; fires outbound webhook only if configured             |

## Application to Training vs. Inference Workloads

|                        | Training workloads (batch GPU jobs)                                | Inference serving (online GPU pods)                              |
|------------------------|--------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Failure mode</b>    | Hard stop at job launch or checkpoint write                        | Silent 401/503 degradation under live traffic                    |
| <b>Primary secrets</b> | Model registry (NGC, HF), S3 checkpoints, W&B/MLflow tracking keys | LLM API keys, TLS certs for endpoints, feature store credentials |

|                            |                                                          |                                                         |
|----------------------------|----------------------------------------------------------|---------------------------------------------------------|
| <b>Blast radius</b>        | Entire job — all GPU-hours from launch to failure        | Per-request revenue loss + SLA breach                   |
| <b>Observability gap</b>   | Job marked Failed with no named cause in existing tools  | Error rate spike looks like app bug, not secret drift   |
| <b>Recovery complexity</b> | Restart from checkpoint or re-queue entire job           | Rolling redeploy required under live production traffic |
| <b>MTTP value</b>          | Reduce hours-long diagnostic to sub-60-second root cause | Catch drift before first 401 reaches end users          |

## Why Existing Tools Do Not Solve This

Each category of existing tooling addresses one layer of the chain. None joins them into a unified causal trace.

| Tool / category            | What it does                                                 | Why it misses this problem                                                                                        |
|----------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <b>HashiCorp VSO</b>       | Re-syncs K8s Secrets when they drift from Vault source       | Only covers store→Secret layer. Does not trace Secret→pod causation. Does not work with existing ESO deployments. |
| <b>ArgoCD / GitOps</b>     | Detects when K8s resources diverge from Git                  | Explicitly configured to ignore ESO-managed Secrets by design — creates a monitoring blind spot.                  |
| <b>Datadog / Dynatrace</b> | Surfaces pod health metrics and log anomalies                | Reports symptom (pod degraded). Has no access to Vault or ExternalSecret state. Cannot name the cause.            |
| <b>Wiz / Prisma (CSPM)</b> | Scans static configs for misconfigurations at deploy time    | Point-in-time scan. Does not detect runtime drift after deployment. No Vault→K8s chain awareness.                 |
| <b>Infisical / Doppler</b> | Secret store with K8s operator and pod restart on change     | Addresses the sync problem for their own store only. Does not monitor existing Vault + ESO deployments.           |
| <b>NVSentinel (NVIDIA)</b> | Monitors GPU hardware health and quarantines unhealthy nodes | Hardware layer only. Config/secret causation chain is explicitly out of scope.                                    |

## Deployment Prerequisites and Constraints

The tracer is designed for enterprises where sending cluster telemetry to a SaaS is prohibited — financial services, healthcare, defense-adjacent, and regulated industries running on-prem or air-gapped GPU clusters. All diagnostic data remains within the cluster boundary.

| Requirement                | Notes                                                                            |
|----------------------------|----------------------------------------------------------------------------------|
| Kubernetes version         | 1.24+                                                                            |
| Secret management stack    | HashiCorp Vault (1.12+) + ESO (0.8+) or Vault Secrets Operator                   |
| New Vault permissions      | None — reuses the ServiceAccount already granted to ESO                          |
| New K8s RBAC               | Read on ExternalSecrets, Secrets, Pods; Write on Events and DriftReport CRD only |
| Network egress             | None required — all Vault calls are in-cluster to the Vault service endpoint     |
| GPU operator compatibility | NVIDIA GPU Operator, NVSentinel, and DGX stack deployments — tested              |
| vCluster support           | Works within virtual clusters; host-cluster ClusterSecretStore drift also traced |

## Business Value: MTTP as the Primary Metric

Mean Time to Prevention (MTTP) is the right frame for this product — not MTTR. The goal is not faster recovery after a secret-caused pod crash. It is catching the drift before the pod crashes, or in the first minutes of degradation rather than hours later.

| Scenario                                                         | Without tracing                                                                                              | With Vault→Pod tracing                                                                                                 |
|------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| HF token expired; 72h training job fails at launch               | SRE opens war room; 3–6h diagnostic; job re-queued from zero                                                 | Vault probe detects missing key at T+0; alert with full chain in <60s; ESO refresh triggered before job starts         |
| Vault token rotation not propagated; inference pod returns 401s  | Error rate alert fires; team investigates app code for 2h; no fix found; secret drift discovered by accident | DriftReport event fires at T+2min; chain trace names Vault path, ExternalSecret, and affected pods; redeploy triggered |
| ClusterSecretStore SA token expires; all ESO syncs fail silently | Multiple unrelated pods fail over 24h; each treated as separate incident; ESO never suspected                | Tracer detects SA token expiry at layer 3; single alert covers all downstream affected pods with shared root cause     |

## Next Steps

### To get started

1. Share your current Vault + ESO cluster topology (namespace count, SecretStore vs. ClusterSecretStore deployment, Vault auth method in use)
2. Identify one training namespace and one inference namespace as the pilot scope — the tracer can be scoped to a single namespace for the initial deployment
3. We deploy the Drift Tracer controller in read-only mode for 48 hours and deliver a DriftReport showing every broken or at-risk chain currently in your cluster — no changes made, no data leaves the cluster
4. Review findings with your Platform Engineering and Security teams; enable remediation actions on agreed chains

**Typical time-to-first-DriftReport: under 4 hours from helm install to first actionable diagnostic.**

*Prepared for qualified distribution. All technical claims based on observed behavior of HashiCorp Vault 1.15, ESO 0.9, and Kubernetes 1.28–1.34 in production-equivalent test environments.*